Queue

Queues are another fundamental data structure characterized by their First In, First Out (FIFO) behavior, meaning the first element added to the queue will be the first one to be removed. Below are the essential operations and functions that can be performed on a queue:

1. **Enqueue**
  - **Description**: This operation adds an element to the back of the queue.
  - **Syntax**: `queue.enqueue(element);` or `queue.push(element);`
  - **Example**: If `queue` is of type `int`, `queue.push(5);` will place the integer `5` at the end of the queue.

2. **Dequeue**
  - **Description**: This operation removes the front element from the queue and returns it.
  - **Syntax**: `queue.dequeue();` or `queue.pop();`
  - **Example**: If the queue's front element is `5`, calling `queue.pop();` will remove `5` and move the next element to the front.

3. **Front / Peek**
  - **Description**: This function retrieves the front element of the queue without removing it.
  - **Syntax**: `queue.front();` or `queue.peek();`
  - **Example**: If the front element is `5`, calling `queue.front();` will return `5` without modifying the queue.

4. **IsEmpty**
  - **Description**: This function checks whether the queue is empty, returning a boolean value.
  - **Syntax**: `queue.isEmpty();`
  - **Example**: If the queue is empty, `queue.isEmpty();` will return `true`.

5. **Size**
  - **Description**: This function returns the number of elements currently in the queue.
  - **Syntax**: `queue.size();`
  - **Example**: If there are three elements in the queue, `queue.size();` will return `3`.

6. **Clear**
  - **Description**: This function removes all elements from the queue.
  - **Syntax**: `queue.clear();`
  - **Example**: After calling `queue.clear();`, the queue will be empty.

7. **Copy Constructor and Assignment Operator**
  - **Description**: These functions allow for the creation of a new queue that is a copy of an existing queue or the assignment of one queue to another.

- **Example**: `Queue newQueue(existingQueue);` will create `newQueue` as a copy of `existingQueue`.

Example Code Using Queue in C++
Here's a simple demonstration of queue operations in C++ using the Standard Template Library (STL):

```cpp
#include <iostream>
#include <queue> // Include the queue header
using namespace std;

int main() {
    queue<int> myQueue; // Create a queue of integers

    // Enqueue elements
    myQueue.push(10);
    myQueue.push(20);
    myQueue.push(30);

    // Display the front element
    cout << "Front element: " << myQueue.front() << endl; // Outputs: 10

    // Dequeue the front element
    myQueue.pop();
    cout << "Front element after dequeue: " << myQueue.front() << endl; // Outputs: 20

    // Check the size of the queue
    cout << "Current queue size: " << myQueue.size() << endl; // Outputs: 2

    // Check if the queue is empty
    cout << "Is queue empty? " << (myQueue.empty() ? "Yes" : "No") << endl; // Outputs: No

    // Clear the queue
    while (!myQueue.empty()) {
        myQueue.pop(); // Remove all elements
    }

    cout << "Queue size after clearing: " << myQueue.size() << endl; // Outputs: 0
    return 0;
}
```

Summary

The operations mentioned above provide a clear and efficient interface for managing data in a queue structure, which is particularly useful in various applications such as scheduling tasks, breadth-first search in graphs, and handling requests in a FIFO manner. If you have any questions or need further examples, feel free to ask!